LeetCode HOT 100

PART 15

Dynamic Programming

动态规划

81. 爬楼梯 (No.70) EASY

假设你正在爬楼梯。需要 `n` 阶你才能到达楼顶。

每次你可以爬 `1` 或 `2` 个台阶。你有多少种不同的方法可以爬到楼顶呢?

示例 1:

> **输入:** n = 2
> **输出:** 2
> **解释:** 有两种方法可以爬到楼顶。
> 1. 1 阶 + 1 阶
> 2. 2 阶

示例 2:

> **输入:** n = 3
> **输出:** 3
> **解释:** 有三种方法可以爬到楼顶。
> 1. 1 阶 + 1 阶 + 1 阶
> 2. 1 阶 + 2 阶
> 3. 2 阶 + 1 阶

提示:

- `1 <= n <= 45`

81. 爬楼梯 (No.70) EASY

$dp[i] = opt_{a \in Actions}\{v(a) + dp[f(i,a)]\}$, $i' = f(i,a)$即状态转移方程

TIPS：动态规划 → ①定义状态(所需变量数) ②状态转移方程(枚举跟当前状态相关的所有之前状态) ③初始化边界条件

思路 本题状态所需变量就1个i，状态f[i]表示到第i层的走法。枚举的相关状态就2种，i-1和i-2。初始化f[0]=f[1]=1。

f[i]表示上i层的走法，他可以由i-2层一次上2层，也可以由i-1层一次上1层，至于怎么走到i-2层和i-1层不用管，直接调用f[i-1]和f[i-2]。因此状态转移方程：f[i]=f[i-1]+f[i-2]。

由于每个i只依赖前两个数，因此没必要用f一个数组来存储所有数值，只需要存储前2期的值就可以了，因此使用滚动数组进行状态压缩(这里其实没有数组，就是两个变量a和b)，空间复杂度从$\mathcal{O}(n)$降到$\mathcal{O}(1)$。

一般写代码时容易想到非滚动的写法就先写，写对了以后才容易看出怎么优化，没必要上来就先状态压缩。

时间复杂度：$\mathcal{O}(n)$；空间复杂度：$\mathcal{O}(1)$

Solution 1 – 优化前

```python
class Solution:
    def climbStairs(self, n: int) -> int:
        f = [0] * (n+2)
        f[0], f[1] = 1, 1
        for i in range(2, n+1):
            f[i] = f[i-1] + f[i-2]
        return f[n]
```

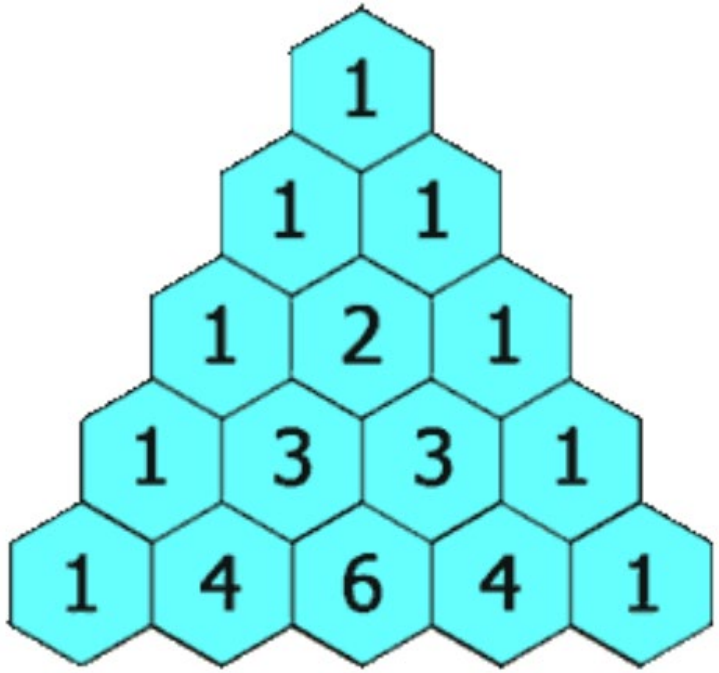
Solution 2 – 优化后

```python
class Solution:
    def climbStairs(self, n: int) -> int:
        a, b = 1, 1  # a表示i-2, b表示i-1
        for _ in range(2, n+1):
            a, b = b, a + b
        return b
```

82. 杨辉三角 (No.118) EASY

给定一个非负整数 `numRows`，生成「杨辉三角」的前 `numRows` 行。

在**「杨辉三角」**中，每个数是它左上方和右上方的数的和。



示例 1:

> **输入：** `numRows = 5`
> **输出：** `[[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]`

示例 2:

> **输入：** `numRows = 1`
> **输出：** `[[1]]`

提示：

- `1 <= numRows <= 30`

82. 杨辉三角 (No.118) EASY

思路

第i行有i个数，一共n行，每行头尾是1。除第1行外，其他行的非头尾元素满足状态转移方程：

$$f[i][j] = f[i-1][j-1] + f[i-1][j]$$

也就是左上角和右上角的元素相加。

```python
class Solution:
    def generate(self, numRows: int) -> List[List[int]]:
        res = [[1] * (i+1) for i in range(numRows)]  # 初始化第1行
        for i in range(1, numRows): # 每行i+1个数，头尾是1
            for j in range(1, i):
                res[i][j] = res[i-1][j-1] + res[i-1][j]
        return res
```

时间复杂度：$\mathcal{O}(n)$；**空间复杂度**：$\mathcal{O}(1)$

83. 打家劫舍 (No.198) MEDIUM

你是一个专业的小偷，计划偷窃沿街的房屋。每间房内都藏有一定的现金，影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统，**如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警。**

给定一个代表每个房屋存放金额的非负整数数组，计算你 **不触动警报装置的情况下** ，一夜之内能够偷窃到的最高金额。

示例 1：

> **输入：** [1,2,3,1]
> **输出：** 4
> **解释：** 偷窃 1 号房屋 (金额 = 1) ，然后偷窃 3 号房屋 (金额 = 3)。
> 偷窃到的最高金额 = 1 + 3 = 4 。

示例 2：

> **输入：** [2,7,9,3,1]
> **输出：** 12
> **解释：** 偷窃 1 号房屋 (金额 = 2), 偷窃 3 号房屋 (金额 = 9), 接着偷窃 5 号房屋 (金额 = 1)。
> 偷窃到的最高金额 = 2 + 9 + 1 = 12 。

提示：

- `1 <= nums.length <= 100`
- `0 <= nums[i] <= 400`

83. 打家劫舍 (No.198) MEDIUM

Solution 1 – 优化前

思路

从左往右偷，f[i]记录当来到第i家能获取的最大金额。对于第i家，枚举相关状态有2个，如果曾到过第i-2家，那可以偷第i家，看看总额多少；或者到过第i-1家，那么第i家就不能偷，那再看看总额多少。这两个总额取较大值就是到了第i家能偷到的最大总额。因此状态转移方程：

$$f[i] = f[i-1] + f[i-2] + nums[i]$$

由于我f的下标i设置为第i家（为了方便处理n=1的情况），所以实际代码中上式的nums[i]其实是nums[i-1]。

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        n = len(nums)
        f = [0] * (n+2)  # f[i]表示从左往右偷到第i家能获取的最高金额
        f[1] = nums[0]  # f[1]初始化，f[0]=0相当于不偷第1家
        for i in range(2, n+1):
            f[i] = max(f[i-1], f[i-2] + nums[i-1])  # f中第i家对应nums中第i-1家
        return f[n]
```

时间复杂度：$\mathcal{O}(n)$；**空间复杂度**：$\mathcal{O}(1)$

83. 打家劫舍 (No.198) MEDIUM

Solution 2 – 优化后

思路

由于f[i]只依赖于前两期的值，可以用滚动数组优化。改成range(1,n)以后注意下标对应关系。

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        n = len(nums)
        a, b = 0, nums[0]
        for i in range(1, n):
            a, b = b, max(a + nums[i], b)
        return b
```

时间复杂度： $\mathcal{O}(n)$；**空间复杂度：** $\mathcal{O}(1)$

84. 完全平方数 (No.279) MEDIUM

给你一个整数 `n` ，返回 *和为 `n` 的完全平方数的最少数量* 。

完全平方数 是一个整数，其值等于另一个整数的平方；换句话说，其值等于一个整数自乘的积。例如，`1`、`4`、`9` 和 `16` 都是完全平方数，而 `3` 和 `11` 不是。

示例 1：

> **输入：** n = `12`
> **输出：** 3
> **解释：** `12 = 4 + 4 + 4`

示例 2：

> **输入：** n = `13`
> **输出：** 2
> **解释：** `13 = 4 + 9`

提示：

- $1 <= n <= 10^4$

84. 完全平方数 (No.279) MEDIUM

思路

前面的题枚举都可以直接列出，因为只依赖之前2期的状态，这道题对于数字n而言，它依赖之前n-1期的状态，因此枚举需要用循环实现。用f[i]表示数字i所需完全平方数的最少数量，则状态转移方程：

$$f[i] = \min_{1 \le j^2 \le i} \{f[i - j * j]\} + 1$$

这表示对于每个i，最少完全平方数数量="i-(某个完全平方数)"的所需数量+1，加的1就是枚举的这个完全平方数。i从1遍历到n，这样i-j*j保证一定是算出来过的数值，符合动态规划无后效性和最优子结构性质。

时间复杂度：$\mathcal{O}(n\sqrt{n})$

空间复杂度：$\mathcal{O}(n)$

```python
class Solution:
    def numSquares(self, n: int) -> int:
        f = [inf] * (n+2)  # f[i]表示数字i对应完全平方数的最少数量
        f[0] = 0
        for i in range(1, n+1):
            j = 1
            # 枚举所有完全平方数
            while j * j <= i:
                f[i] = min(f[i], f[i-j*j] + 1)
                j += 1
        return f[n]
```

85. 零钱兑换 (No.322) MEDIUM

给你一个整数数组 `coins` ，表示不同面额的硬币；以及一个整数 `amount` ，表示总金额。

计算并返回可以凑成总金额所需的 **最少的硬币个数** 。如果没有任何一种硬币组合能组成总金额，返回 `-1` 。

你可以认为每种硬币的数量是无限的。

示例 1：

> **输入：** coins = `[1, 2, 5]`, amount = `11`
> **输出：** `3`
> **解释：** 11 = 5 + 5 + 1

示例 2：

> **输入：** coins = `[2]`, amount = `3`
> **输出：** -1

示例 3：

> **输入：** coins = [1], amount = 0
> **输出：** 0

提示：

- `1 <= coins.length <= 12`
- `1 <= coins[i] <=` $2^{31} - 1$
- `0 <= amount <=` 10^4

85. 零钱兑换 (No.322) MEDIUM

思路

这题跟[84.完全平方数]几乎一样，只不过枚举的不再是从1到根号i，而是直接枚举可选的硬币面额。因此状态转移方程：$f[i] = \min_{j \in coins} \{f[i-j]\} + 1$。枚举的时候注意，面额不能超过i的，否则i-j下标越界。这题其实用回溯“选或不选”的模板也可以写，本质上跟[58.总和组合]一模一样。

```
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        f = [inf] * (amount + 2)  # f[i]表示总额为i有多少种硬币组合
        f[0] = 0
        for i in range(amount+1):
            for j in coins:
                if j <= i:  # 硬币面额不能超过总额
                    f[i] = min(f[i], f[i-j] + 1)
        return f[amount] if f[amount] != inf else -1
```

时间复杂度：$\mathcal{O}(n \cdot \text{amount})$，$n$为coins长度；**空间复杂度**：$\mathcal{O}(amount)$

86. 单词拆分 (No.139) MEDIUM

给你一个字符串 `s` 和一个字符串列表 `wordDict` 作为字典。如果可以利用字典中出现的一个或多个单词拼接出 `s` 则返回 `true`。

注意：不要求字典中出现的单词全部都使用，并且字典中的单词可以重复使用。

示例 1：

> **输入：** `s = "leetcode", wordDict = ["leet", "code"]`
> **输出：** `true`
> **解释：** 返回 true 因为 "leetcode" 可以由 "leet" 和 "code" 拼接成。

示例 2：

> **输入：** `s = "applepenapple", wordDict = ["apple", "pen"]`
> **输出：** `true`
> **解释：** 返回 true 因为 "applepenapple" 可以由 "apple" "pen" "apple" 拼接成。
> 注意，你可以重复使用字典中的单词。

示例 3：

> **输入：** `s = "catsandog", wordDict = ["cats", "dog", "sand", "and", "cat"]`
> **输出：** `false`

提示：

- `1 <= s.length <= 300`
- `1 <= wordDict.length <= 1000`
- `1 <= wordDict[i].length <= 20`
- `s` 和 `wordDict[i]` 仅由小写英文字母组成
- `wordDict` 中的所有字符串 **互不相同**

86. 单词拆分 (No.139) MEDIUM

思路

对于s的位置i，假如s[0:j]已经能由wordDict拼出，那只要s[j:i]也是字典中某个单词就可以了。

f[i]表示s[0:i]是否能由wordDict拼出，则状态转移方程：$f[i] = \underset{0 \le j \le i-1}{\text{OR}} \{s[j:i] \in wordDict\}$，$OR$表示只要枚举的多个j有一个为True则结果为True。

下面的代码为了运行更快，其实只要f[i]变为True就可以break内循环了，这里为了代码简洁性没有剪枝。

```
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:
        n = len(s)
        f = [False] * (n+2)  # f[i]表示s[0:i]是否能由wordDict拼出
        f[0] = True
        for i in range(1, n+1):
            for j in range(i):
                f[i] = f[i] or f[j] and s[j:i] in wordDict
        return f[n]
```

时间复杂度：$\mathcal{O}(n^2)$，n为s长度；**空间复杂度**：$\mathcal{O}(n)$

87. 最长递增子序列 (No.300) MEDIUM

给你一个整数数组 `nums` ，找到其中最长严格递增子序列的长度。

子序列 是由数组派生而来的序列，删除（或不删除）数组中的元素而不改变其余元素的顺序。例如，`[3,6,2,7]` 是数组 `[0,3,1,6,2,2,7]` 的子序列。

示例 1：

> **输入：** nums = [10,9,2,5,3,7,101,18]
> **输出：** 4
> **解释：** 最长递增子序列是 [2,3,7,101]，因此长度为 4 。

示例 2：

> **输入：** nums = [0,1,0,3,2,3]
> **输出：** 4

示例 3：

> **输入：** nums = [7,7,7,7,7,7,7]
> **输出：** 1

提示：

- `1 <= nums.length <= 2500`
- `-10^4 <= nums[i] <= 10^4`

进阶：

- 你能将算法的时间复杂度降低到 `O(n log(n))` 吗？

87. 最长递增子序列 (No.300) MEDIUM

Solution 1 – 动态规划

思路

f[i]表示以nums[i]为末尾的最长递增子序列长度，那么对于每个i，要枚举i之前的所有j，如果nums[i]>nums[j]，那么可以把nums[i]接在nums[j]后面，LIS长度+1。最后对整个f取最大值。

状态转移方程：$f[i] = \max_{0 \le j \le i-1} \{f[j]\} + 1$。

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        f = [1] * n  # f[i]表示以nums[i]为末尾的LIS长度
        for i in range(1, n):
            for j in range(i):
                if nums[i] > nums[j]:
                    f[i] = max(f[i], f[j] + 1)
        return max(f)
```

时间复杂度：$\mathcal{O}(n^2)$，n为s长度；**空间复杂度**：$\mathcal{O}(n)$

87. 最长递增子序列 (No.300) MEDIUM

Solution 2 – 贪心+二分查找

思路

直观来说，每当遍历当一个数字，我们希望它保持递增的前提下尽可能小，这更有利于后续创造更长的递增子序列。

用f[i]表示：长度为i+1的所有递增子序列中最小的末尾数字。

当f为空或者遍历的num大于f末尾，就继续延长f；否则用二分查找找到num在f中应插入的位置i，i+1表示num作为当前某个递增子序列的末尾，这个递增子序列最长是多少。假如后续还有更长的递增子序列，新来的元素会不断替换掉i之后的所有元素，直到可以继续延长f。

这里的二分查找可以直接套用[63.搜索插入位置]的模板。

时间复杂度：$\mathcal{O}(n\log n)$；**空间复杂度**：$\mathcal{O}(n)$

```python
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        def binarySearch(f, target):
            left, right = 0, len(f)-1
            while left <= right:
                mid = (left + right) // 2
                if f[mid] < target:
                    left = mid + 1
                else:
                    right = mid - 1
            return left

        f = []  # f[i]表示长度为i+1的所有递增子序列中，最小的末尾数字
        for num in nums:
            # 如果f为空或者num比f末尾大，num加入f
            if not f or num > f[-1]:
                f.append(num)
            # 否则二分查找找到num插入的位置放入num
            else:
                i = binarySearch(f, num)
                f[i] = num
        return len(f)
```

88. 乘积最大子数组 (No.152) MEDIUM

给你一个整数数组 `nums` ，请你找出数组中乘积最大的非空连续 子数组（该子数组中至少包含一个数字），并返回该子数组所对应的乘积。

测试用例的答案是一个 **32-位** 整数。

请注意，一个只包含一个元素的数组的乘积是这个元素的值。

示例 1:

> **输入：** nums = [2,3,-2,4]
> **输出：** 6
> **解释：** 子数组 [2,3] 有最大乘积 6。

示例 2:

> **输入：** nums = [-2,0,-1]
> **输出：** 0
> **解释：** 结果不能为 2, 因为 [-2,-1] 不是子数组。

提示：

- `1 <= nums.length <= 2 * 10^4`
- `-10 <= nums[i] <= 10`
- `nums` 的任何子数组的乘积都 **保证** 是一个 **32-位** 整数

88. 乘积最大子数组 (No.152) MEDIUM

思路

这题和上一题类似，也会想到用f[i]来记录以nums[i]结尾的乘积最大子数组，但由于有负数的存在，可能i-1处的最小值，乘上nums[i]会变成位置i处的最大值。因此需要f_max和f_min两个数组来存储以nums[i]结尾的“乘积最大子数组”和“乘积最小子数组”。状态转移方程可以从代码中显式地看出，每次比较nums[i],f_min[i-1]*nums[i],f_max[i-1]*nums[i]三者大小。

```
class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        n = len(nums)
        f_min, f_max = [nums[0]] * n, [nums[0]] * n
        for i in range(1, n):
            f_min[i] = min(nums[i], f_min[i-1] * nums[i], f_max[i-1] * nums[i])
            f_max[i] = max(nums[i], f_min[i-1] * nums[i], f_max[i-1] * nums[i])
        return max(f_max)
```

时间复杂度：$\mathcal{O}(n)$；**空间复杂度**：$\mathcal{O}(n)$

89. 分割等和子集 (No.416) MEDIUM

给你一个 **只包含正整数** 的 **非空** 数组 `nums` 。请你判断是否可以将这个数组分割成两个子集，使得两个子集的元素和相等。

示例 1：

> **输入：** `nums = [1,5,11,5]`
> **输出：** `true`
> **解释：** 数组可以分割成 `[1, 5, 5]` 和 `[11]` 。

示例 2：

> **输入：** `nums = [1,2,3,5]`
> **输出：** `false`
> **解释：** 数组不能分割成两个元素和相等的子集。

提示：

- `1 <= nums.length <= 200`
- `1 <= nums[i] <= 100`

89. 分割等和子集 (No.416) MEDIUM

Solution 1 – 优化前

思路

首先如果能够等和分割，那么数字总和summ必须为偶数。所以总和奇数直接返回False。偶数的情况下，如果要等和分割，那么应该选出若干数，总和为summ//2，这就变成了一个0-1背包问题。之前[84.完全平方数]和[85.零钱兑换]是完全背包，同一个元素可以重复取。0-1背包由于不能重复取相同元素，状态空间要添加一个位置pos的维度。f[i][j]表示nums[0:i+1]是否能够凑出总和为j的组合。那么状态转移方程：f[i][j] = f[i-1][j] or nums[i]≤j and f[i-1][j-nums[i]]。

这个状态转移其实就是回溯中“选或不选”nums[i]的逻辑。如果不选，那它和前面i-1个数能不能凑够总额j同真假；如果选，那么首先满足nums[i]比当前总和j小，其次既然选了nums[i]，那么前面i-1个数要能凑够j-nums[j]。

```python
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        summ = sum(nums)
        if summ % 2:  # 总和奇数，不可能等和分割
            return False
        m = summ // 2  # 0-1背包问题，求出是否有总和为m的组合
        n = len(nums)
        # f维度n*m，f[i][j]表示nums[0:i+1]是否能总和为j
        f = [[False] * (m+1) for _ in range(n)]
        f[0][0] = True
        for i in range(1, n):  # 遍历所有nums
            for j in range(m+1):  # 遍历所有总和
                # 0-1背包：选或不选
                f[i][j] = f[i-1][j] or nums[i] <= j and f[i-1][j-nums[i]]
        return f[n-1][m]
```

时间复杂度：$\mathcal{O}(mn)$，m为数字总和，n为nums长度；**空间复杂度**：$\mathcal{O}(mn)$

89. 分割等和子集 (No.416) MEDIUM

Solution 2 – 优化后

思路

状态转移方程“f[i][j] = f[i-1][j] or nums[i]≤j and f[i-1][j-nums[i]]”表明，对于f[i][j]，只依赖前1期的f[i-1][*]，因此可以对i所在的维度进行状态压缩。由于j依赖于比他小的j-nums[i]这一状态，因此当用一维滚动数组维护f以后，必须倒序遍历j，否则当j需要j-nums[i]的时候，本轮的f已经被刷新了，调用的并不是压缩前的f[i-1][j-nums[i]]，而变成了压缩前的f[i][j-nums[i]]，那就出错了。这题是0-1背包动态规划的一个经典模板。

```python
class Solution:
    def canPartition(self, nums: List[int]) -> bool:
        summ = sum(nums)
        if summ % 2:  # 总和奇数，不可能等和分割
            return False
        m = summ // 2  # 0-1背包问题，求出是否有总和为m的组合
        n = len(nums)
        # f维度n*m，f[i][j]表示nums[0:i+1]是否能总和为j
        f = [False] * (m+1)  # 用一维数组f[j]滚动表示之前的f[i][j]
        f[0] = True
        for i in range(1, n):  # 遍历所有nums
            for j in range(m, -1, -1):  # 遍历所有总和
                # 状态转移依赖之前的状态，滚动数组要倒序遍历
                f[j] = f[j] or nums[i] <= j and f[j-nums[i]]
        return f[m]
```

时间复杂度：$\mathcal{O}(mn)$，m为数字总和，n为nums长度；**空间复杂度**：$\mathcal{O}(m)$

90. 最长有效括号 (No.32) HARD

给你一个只包含 `'('` 和 `')'` 的字符串，找出最长有效（格式正确且连续）括号 子串 的长度。

左右括号匹配，即每个左括号都有对应的右括号将其闭合的字符串是格式正确的，比如 `"(()())"`。

示例 1：

> **输入：** s = "(()"
> **输出：** 2
> **解释：** 最长有效括号子串是 "()"

示例 2：

> **输入：** s = ")()())"
> **输出：** 4
> **解释：** 最长有效括号子串是 "()()"

示例 3：

> **输入：** s = ""
> **输出：** 0

提示：

- $0 <= s.length <= 3 * 10^4$
- `s[i]` 为 `'('` 或 `')'`

90. 最长有效括号 (No.32) HARD

Solution 1 – 动态规划+栈

思路

借助栈实现类似DP的操作，f[i]表示以i为末尾的最长有效括号长度。遇到左括号下标入栈；遇到右括号，如果栈中有左括号匹配，栈顶left弹出，left到i保证是匹配好的部分（否则遍历到i栈顶不会是left这个位置），那么left前一个位置对应的最长有小括号子串，拼上left到i这一段就是f[i]。

由于遇到i时其实直接取栈顶，不需要枚举，所以流程上和之前的DP有点不同，但本质上仍是用DP，因为借助了已有的子问题的解来求当前解。

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(n)$

```python
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        n = len(s)
        f = [0] * n  # f[i]表示以s[i]为末尾的最长有效括号长度
        stack = []
        for i in range(n):
            if s[i] == '(':
                stack.append(i)  # 左括号下标入栈
            else:
                if stack:  # 有左括号匹配
                    left = stack.pop()
                    # 能看到left处的左括号，left和i之间已经是有效括号
                    f[i] = f[left-1] + (i - left + 1)
        return max(f) if f else 0
```

90. 最长有效括号 (No.32) HARD

Solution 2 – 双指针

思路

双指针可以将空间复杂度降到$\mathcal{O}(1)$。left和right分别表示左右括号的计数，当右括号溢出时重置，当left=right时出现有效括号，此时更新res。不过从左往右可能会漏掉“(()”这种左括号冗余的情况，因此还要从右往左遍历一遍。

从右往左遍历时要反过来，遇到右括号left+1，遇到左括号right+1，这个通过传入不同的left_bracket实现。

时间复杂度：$\mathcal{O}(n)$

空间复杂度：$\mathcal{O}(1)$

```python
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        def findMaxLength(s, left_bracket):
            # 从左往右遍历找最长有效括号长度，自定义"左括号"left_bracket
            res = 0
            left, right = 0, 0
            for char in s:
                if char == left_bracket:
                    left += 1
                else:
                    right += 1
                    if right > left:  # 右括号溢出，重置
                        left, right = 0, 0
                    elif left == right:  # 有效括号产生
                        res = max(res, right * 2)
            return res

        return max(findMaxLength(s, '('), findMaxLength(reversed(s), ')'))
```